

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-7:
- klasa algorytmu genetycznego

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025

Jakich parametrów składowych potrzebujemy?

- Informacji kiedy ma się zakończyć praca algorytmu:
 - Jako, że w naszym przypadku nie mamy żadnych przypuszczeń co do najlepszej wartości rozwiązywanego problemu, będziemy sprawdzać zbieżność najlepszego wyniku z każdej populacji.
 - Drugim warunkiem zatrzymania będzie ograniczenie maksymalnej liczby utworzonych populacji.
- Zmiennej przechowującej obiekt klasy Tpopulation (spoiler: lub jej adres...).
- Jako, że nowa populacja będzie tworzona na bazie wcześniejszej populacji potrzebujemy do przechowywania poprzedniej populacji (obektu klasy Tpopulation).

TAlgorithm.h

```
1  #pragma once
2
3  #include "TPopulation.h"
4
5  class TAlgorithm
6  {
7      unsigned int stop_max_population_count;
8      unsigned int stop_min_improvement_proc;
9
10     TPopulation* wsk_population_pres = nullptr;
11     TPopulation* wsk_population_prev = nullptr;
12
13
14
15
16
17
18
19
20
21
22
23
24
25 };
```

Jakich funkcji składowych potrzebujemy?

- Konstruktor, w którym **dynamicznie** tworzącego populację;
- Destruktora, w którym usunięte z pamięci zostaną **dynamicznie** utworzone populacje;
- Funkcji, która wywoła uruchomienie algorytmu – poszukiwanie wartości optymalnej rozważanego zagadnienia;
- **Prywatnych** funkcji sprawdzających warunki zakończenia pracy algorytmu.

```
1  #pragma once
2
3  #include "TPopulation.h"
4
5  class TAlgorithm
6  {
7      unsigned int stop_max_population_count;
8      unsigned int stop_min_improvement_proc;
9
10     TPopulation* wsk_population_pres = nullptr;
11     TPopulation* wsk_population_prev = nullptr;
12
13     public:
14         TAlgorithm(unsigned int candidates_count = 10,
15                     unsigned int max_population_count = 20,
16                     unsigned int min_improvement_proc = 3);
17
18         ~TAlgorithm();
19
20         void run();
21
22     private:
23         bool is_stop();
24         bool is_max_population();
25         bool is_min_improvement();
26     };
27
```

```
idate.cpp  TPopulation.cpp  TAlgorithm.cpp  X
2024      ↓ TAlgorithm      ~TAlgor

1  #include <iostream>
2  #include <math.h>
3
4  #include "TAlgorithm.h"
5
6  using namespace std;
7
8  TAlgorithm::TAlgorithm(unsigned int candidates_count,
9      unsigned int max_population_count,
10     unsigned int min_improvement_proc)
11  {
12      stop_max_population_count = max_population_count;
13      stop_min_improvement_proc = min_improvement_proc;
14
15      wsk_population_pres = new TPopulation{ candidates_count };
16  }
17
18  TAlgorithm::~TAlgorithm()
19  {
20      delete wsk_population_pres;
21      delete wsk_population_prev;
22
23      cout << "Usunięto obiekt klasy TAlgorithm";
24  }
```

Konstruktor

Linia 12 – 13: zapisanie wartości do parametrów składowych.

Linia 15: **dynamiczne**, tzn. w trakcie działania programu, utworzenie obiektu klasy *TPopulation*. Prawa strona równania zwraca **wskaźnik** do obiektu. **Obiekt nie ma nazwy** za pomocą której można by się do niego odwołać – znamy jedynie jego adres w pamięci (poprzez wskaźnik).

Destruktor

Linia 18-24: usunięcie z pamięci dynamicznie utworzonych obiektów klasy *TPopulation*.

```
26 void TAlgorithm::run()
27 {
28     bool stop = false;
29
30     while (!wsk_population_prev || !stop)
31     {
32         //(*wsk_population_pres).calculate();
33         wsk_population_pres->calculate();
34
35         cout << "==" Population #" << wsk_population_pres->get_id();
36         cout << " || best val: " << wsk_population_pres->get_best_val() << endl;
37
38         stop = is_stop();
39
40         if (!stop)
41         {
42             unsigned int candidates_count = wsk_population_pres->get_candidates_count();
43
44             delete wsk_population_prev;
45             wsk_population_prev = wsk_population_pres;
46
47             // chwilowe rozwiazanie - tworzenie kolejnej losowej populacji
48             wsk_population_pres = new TPopulation{ candidates_count };
49         }
50
51         if (wsk_population_pres->get_id() == 25) return;
52     }
53 }
54
```

Linia 30: algorytm działa dopóki spełniony jest jeden z warunków – co istotne **warunki są sprawdzane od lewej strony!** Jeśli pierwszy warunek jest spełniony drugi nie jest sprawdzany. Pierwszy warunek jest spełniony, gdy obliczenia mają zostać wykonane dla pierwszej populacji (wskaźnik do poprzedniej populacji jest pusty). W przeciwnym przypadku sprawdzane są warunki zatrzymanie algorytmu – ostateczna decyzja o zatrzymaniu algorytmu przechowywana jest w zmiennej pomocniczej *stop*.

```
26 void TAlgorithm::run()
27 {
28     bool stop = false;
29
30     while (!wsk_population_prev || !stop)
31     {
32         //(*wsk_population_pres).calculate();
33         wsk_population_pres->calculate();
34
35         cout << "==" Population #" << wsk_population_pres->get_id();
36         cout << " || best val: " << wsk_population_pres->get_best_val() << endl;
37
38         stop = is_stop();
39
40         if (!stop)
41         {
42             unsigned int candidates_count = wsk_population_pres->get_candidates_count();
43
44             delete wsk_population_prev;
45             wsk_population_prev = wsk_population_pres;
46
47             // chwilowe rozwiazanie - tworzenie kolejnej losowej populacji
48             wsk_population_pres = new TPopulation{ candidates_count };
49         }
50
51         if (wsk_population_pres->get_id() == 25) return;
52     }
53 }
54
```

Linia 32 - 33: dwa sposoby odwołania się do obiektu i jego funkcji składowych. Wywołanie funkcji populacji, która oblicza wartości osobników.

Linia 35 – 36: jako że obliczenia mogą trwać długo na ekranie wyświetlane są informacje o aktualnym postępie pracy algorytmu.


```
26 void TAlgorithm::run()
27 {
28     bool stop = false;
29
30     while (!wsk_population_prev || !stop)
31     {
32         /*(*wsk_population_pres).calculate();
33         wsk_population_pres->calculate();
34
35         cout << "== Population #" << wsk_population_pres->get_id();
36         cout << " || best val: " << wsk_population_pres->get_best_val() << endl;
37
38         stop = is_stop();
39
40         if (!stop)
41         {
42             unsigned int candidates_count = wsk_population_pres->get_candidates_count();
43
44             delete wsk_population_prev;
45             wsk_population_prev = wsk_population_pres;
46
47             // chwilowe rozwiazanie - tworzenie kolejnej losowej populacji
48             wsk_population_pres = new TPopulation{ candidates_count };
49         }
50
51         if (wsk_population_pres->get_id() == 25) return;
52     }
53 }
```

Linia 42: zmienna pomocnicza przechowująca informację o liczebności populacji.

Linia 44: usunięcie obiektu znajdującego się pod adresem wskazanym przez wskaźnik – usunięcie starej populacji.

Linia 45: należy zrobić miejsce dla nowej populacji, dlatego adres obecnej populacji zapisujemy w parametrze *wsk_population_prev*.

Linia 48: dla celów testowych tworzymy nową populację starą metodą – poprzez losowanie nowych osobników. Gdy stwierdzimy, że na tym etapie algorytm działa poprawnie zajmiemy się poprawnym tworzeniem nowej populacji – na bazie poprzedniej.

```
77 bool TAlgorithm::is_max_population()
78 {
79     unsigned int _id_pres = wsk_population_pres->get_id();
80     return (_id_pres == stop_max_population_count);
81 }
82
83 bool TAlgorithm::is_min_improvement()
84 {
85     TPopulation population_pres = (*wsk_population_pres);
86     TPopulation population_prev = (*wsk_population_prev);
87
88     double best_val_pres = population_pres.get_best_val();
89     double best_val_prev = population_prev.get_best_val();
90
91     int improvement = fabs(best_val_prev - best_val_pres) / best_val_prev * 100;
92     return (improvement <= stop_min_improvement_proc);
93 }
```

Dla dwóch warunków zatrzymania algorytmu stworzymy dwie osobne funkcje pomocnicze. Przy dużej liczbie zaimplementowanych warunków zatrzymania łatwo będzie wybrać, z których finalnie chce się skorzystać – poprzez wywołanie dedykowanych funkcji pomocniczych.


```
56 bool TAlgorithm::is_stop()
57 {
58     bool stop = false;
59
60     if (!wsk_population_prev) // rozpatrywana pierwsza populacja
61     {
62         stop = is_max_population();
63     }
64     else
65     {
66         bool stop_case1 = is_max_population();
67         bool stop_case2 = is_min_improvement();
68         stop = stop_case1 || stop_case2;
69         // cout << "stop_case1: " << stop_case1 << endl;
70         // cout << "stop_case2: " << stop_case2 << endl;
71     }
72
73     // cout << "stop: " << stop << endl;
74     return stop;
75 }
76
```

Dla pierwszej populacji nie
można sprawdzić zbieżności
z poprzednią populacją

Linia 42: zmienna pomocnicza przechowująca informacje o liczebności populacji.

```
42 | | | unsigned int candidates_count = wsk_population_pres->get_candidates_count();
```

```
79 | | | unsigned int _id_pres = wsk_population_pres->get_id();
```

```
88 | | | double best_val_pres = population_pres.get_best_val();  
89 | | | double best_val_prev = population_prev.get_best_val();
```

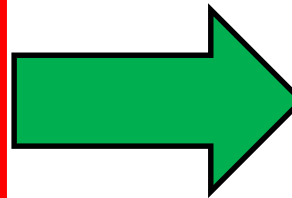
Należy dodać do klasy *TPopulation* brakujące funkcje

```
main.cpp  TCandidate.h  TPopulation.h  TAlgorithm.h
AG2024    TPopulation
1  #pragma once
2
3  #include <vector>
4  #include "TCandidate.h"
5
6  class TPopulation
7  {
8      static unsigned int    population_count;
9      unsigned int          _id;
10     unsigned int           candidate_count;
11     std::vector<TCandidate> candidates;
12     double                 best_val = 0;
13
14     public:
15         TPopulation(unsigned int cands_count = 10);
16         void         calculate();
17         TCandidate   get_best_candidate();
18
19         unsigned int get_id() { return _id; }
20         unsigned int get_candidates_count() { return candidate_count; }
21         double       get_best_val() { return best_val; }
22
23         void         info();
24     };
25
```

Funkcja celu: $x_1^2 + x_2$
 x_1 [0, 100, 1]
 x_2 [0, 10, 1]

```
main.cpp  TCandidate.h  TPopulation.h  TAlgorithm.h
AG2024    (Globalny zasięg)

1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4
5  #include "TAlgorithm.h"
6
7  using namespace std;
8
9
10 int main()
11 {
12     srand(time(0));
13
14     unsigned int candidates_count = 5;
15     unsigned int max_population_count = 20;
16     unsigned int min_improvement_proc = 2;
17
18     TAlgorithm task{ candidates_count,
19                     max_population_count,
20                     min_improvement_proc };
21     task.run();
22
23     return 0;
24 }
```



Konsola debugowania programu Microsoft Visual Studio

```
== Population #0 || best val: 7059
== Population #1 || best val: 10000
== Population #2 || best val: 6894
== Population #3 || best val: 10010
== Population #4 || best val: 8100
== Population #5 || best val: 1852
== Population #6 || best val: 6242
== Population #7 || best val: 7060
== Population #8 || best val: 8655
== Population #9 || best val: 5191
== Population #10 || best val: 9412
== Population #11 || best val: 4357
== Population #12 || best val: 6250
== Population #13 || best val: 7406
== Population #14 || best val: 7406
Usunięto obiekt klasy TAlgorithm
```